

Introduction to Particle Filtering

Jeff Steward

January 28, 2013

1 Introduction to filters

A “filter” in this context is used to describe a mathematical process that removes an unwanted component from data. This could be thought of as removing noise from data, removing some frequencies of the data (through a Fourier-transform, for example), or smoothing the data.

For our purposes, we will take a filter to be software that attempts to remove noise from data. We can look at this either from the perspective of stochastic differential equations or from a probability perspective. In this tutorial, I will attempt to show both approaches as they are equally useful.

1.1 An example

We will consider the following situation. First, suppose we have a model of how a system evolves that includes a random component. For example, we could assume that there is a car that is driving around in a parking lot with the driver constantly choosing random acceleration. The model for this could look like:

$$\begin{aligned} dx/dt &= u \\ dy/dt &= v \\ du/dt &= a_x \\ dv/dt &= a_y \end{aligned}$$

Where (x, y) is the position and (u, v) are the x and y component of acceleration. We could assume that (a_x, a_y) is “random” - that is, it is changing all the time.

This description, while intuitive, is unfortunately insufficient mathematically. What is meant by “changing all the time”? We therefore use the more complicated-looking stochastic differential equation model

$$\begin{aligned} dx_t &= u_t dt \\ dy_t &= v_t dt \\ du_t &= a_x dB_t \\ dv_t &= a_y dB_t \end{aligned}$$

Here, $\mathbf{x}_t = (x_t, y_t, u_t, v_t)^\top$ is a *stochastic process*, meaning that at each time t_k , x_k, y_k, u_k and v_k are random variables. While a random variable is a function that maps an event to a probability, a stochastic process is a function that maps a time index to a random variable. The B here stands for Brownian motion, and dB_t means that the variance of the random variables u_t and v_t grows by an amount $a_x^2 dt$ and $a_y^2 dt$ in time dt .

To say this in a more understandable way, the SDE

$$dx_t = \mu(x_t, t)dt + \sigma(x_t, t)dB_t$$

means that in a small amount of time δt , the random variable $x_{t+\delta t}$ will have changed its mean, on average, by $\mu(x_t, t)\delta t$ when compared to x_t . To be precise, the change in mean of $x_{t+\delta t}$ will be normally distributed with mean $\mu(x_t, t)\delta t$ and variance $\sigma^2(x_t, t)\delta t$.

This interpretation leads us to the discrete version of the system above. We discretize the equation at N_t regularly spaced points in time $t = (0, t_f)$, and we call $(x_k^d, y_k^d, u_k^d, v_k^d)$ our discrete values that approximate the value of (x_k, y_k, u_k, v_k) at time $t_k = k\Delta t$, where $\Delta t = t_f/N_t$. From here on, we will drop the superscript d on the discrete variables as it is tedious to write and we are most interested in the discrete case.

If we approximate $dx_k = x_k - x_{k-1}$, we find

$$\begin{aligned} x_k &= x_{k-1} + u_k \Delta t \\ y_k &= y_{k-1} + v_k \Delta t \\ u_k &= u_{k-1} + \eta_{x_k} \Delta t \\ v_k &= v_{k-1} + \eta_{y_k} \Delta t \end{aligned}$$

Here, η_{x_k} and η_{y_k} are normally distributed variables with mean 0 and variances of a_x^2 and a_y^2 , respectively. Since x_k and y_k involve u_k and v_k from the same time step, we substitute in the value for u_k and v_k in those expressions, giving

$$\begin{aligned} x_k &= x_{k-1} + u_{k-1} \Delta t + \eta_{x_k} \Delta t^2 \\ y_k &= y_{k-1} + v_{k-1} \Delta t + \eta_{y_k} \Delta t^2 \\ u_k &= u_{k-1} + \eta_{x_k} \Delta t \\ v_k &= v_{k-1} + \eta_{y_k} \Delta t \end{aligned}$$

We now state our system in terms of probability. We recall that for a normal distribution $X \sim \mathcal{N}(\mu, \sigma^2)$, the variable $Y = aX + b \sim \mathcal{N}(a\mu + b, a^2\sigma^2)$. Since the noise term is normally distributed, we see that

$$\begin{aligned} x_k &\sim \mathcal{N}(x_{k-1} + u_{k-1} \Delta t, a_{x_k}^2 \Delta t^4) \\ y_k &\sim \mathcal{N}(y_{k-1} + v_{k-1} \Delta t, a_{y_k}^2 \Delta t^4) \\ u_k &\sim \mathcal{N}(u_{k-1}, a_{x_k}^2 \Delta t^2) \\ v_k &\sim \mathcal{N}(v_{k-1}, a_{y_k}^2 \Delta t^2) \end{aligned}$$

Since the noise terms η_x and η_y are now present, we cannot hope to “solve” the system in the sense of finding “the” solution as we can for an ordinary differential equation or a partial differential equation. Instead, we will seek

the probability density of the possible states at each step in time. In order to find this probability density function, we could analyze the problem directly if the system is relatively simple, or we could discretize the problem and run a numerical version of the system several times to learn from the statistics of our runs.

When we run the system many times, we call a collection of runs of a system an “ensemble” and each individual run a “member” or, in the case of particle filters, a “particle.” From the ensemble, we will be able to retrieve empirical statistics that will give us the approximate probability density function of states.

1.2 Observations

We now introduce the more realistic case where instead of knowing the state of the system directly, we have observations of the system that have noise in them. For example, imagine that in the car problem considered above you have some measurements of the x and y location of the car at each time step k . The machine you are getting these observations from is not perfect and the observations have some error in them. For the sake of simplicity, we wish to consider the case where the errors are normally distributed. Let’s also assume we know the mean and variance of these errors. In reality, one would of course hope the mean was zero (i.e. the measurements are unbiased) and the variance of the error is small.

We want to take these observations and estimate what the state of the system should be. We will use our model of the system (which we will trust is an accurate depiction of the system) and the known statistics of the state change and observation error in order to make a prediction of the state $\mathbf{x}_k$ at each time step $k = 1, 2, \dots, N_t$. This prediction will depend on the quality of the observations we have, i.e. on the variance of the observation errors.

We create a model of the observations as a function of the state, which in this case is very simple.

$$\begin{aligned} z_{k_x} &= x_k + n_{k_x} \\ z_{k_y} &= y_k + n_{k_y} \end{aligned}$$

Here, $\mathbf{z}_k = (z_{k_x}, z_{k_y})^\top$ are our imperfect observations. x_k and y_k represent the true state, which it is assumed our state model accurately represents. n_{k_x} and n_{k_y} are Gaussian variables with mean 0 and variance R_k .

Our task is to estimate $\mathbf{x}_k$ based on the observations $\mathbf{z}_k$, our model of the system, knowing the variance of the state change Q_k , our model of the observations, and the variance of our observations R_k . We want to do this at each time step k , although we assume we only have the information for time steps $i \leq k$. This is a very important problem in many real world situations.

1.3 Generalization

We now generalize the problem described above. Suppose that we have a state model

$$\mathbf{x}_k = f_k(\mathbf{x}_{k-1}, \mathbf{v}_{k-1})$$

where $\mathbf{x}_k, \mathbf{x}_{k-1} \in \mathbb{R}^{N_x}$ are the state variables at time k and $k-1$, respectively, $\mathbf{v}_{k-1} \in \mathbb{R}^{N_v}$ is multivariate Gaussian noise with mean $\{\mathbf{0}\}_{\mathbb{R}^{N_v}}$ and covariance $Q_k \in \mathbb{R}^{N_v} \times \mathbb{R}^{N_v}$. Note that the covariance of the noise can change with k . f_k can be a non-linear function, and can also change at each time step k .

Suppose we also have an observation model

$$\mathbf{z}_k = h_k(\mathbf{x}_k, \mathbf{n}_k)$$

where $\mathbf{z}_k \in \mathbb{R}^{N_z}$ is the observation at time step k , $\mathbf{x}_k \in \mathbb{R}^{N_x}$ is the state at time k , and $\mathbf{n}_k$ is multivariate Gaussian noise with mean $\{\mathbf{0}\}_{\mathbb{R}^{N_n}}$ and covariance $R_k \in \mathbb{R}^{N_n} \times \mathbb{R}^{N_n}$. h_k is a possibly non-linear function. Both R_k and h_k are allowed to vary at each time step k .

In this framework, our example problem becomes

$$\mathbf{x}_k = f_k(\mathbf{x}_{k-1}, \mathbf{v}_{k-1}) = F\mathbf{x}_{k-1} + G\mathbf{v}_{k-1}$$

where $\mathbf{x}_k = (x, y, u, v)^\top$, $\mathbf{v}_{k-1} = (\eta_x, \eta_y)^\top$ is a multivariate Gaussian variable with mean $\{\mathbf{0}\}_{\mathbb{R}^2}$ and the covariance matrix Q_k is

$$Q_k = \begin{pmatrix} a_x^2 & 0 \\ 0 & a_y^2 \end{pmatrix}$$

F is a matrix where

$$F = \begin{pmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

and G , also a matrix, is given by

$$G = \begin{pmatrix} \Delta t^2 & 0 \\ 0 & \Delta t^2 \\ \Delta t & 0 \\ 0 & \Delta t \end{pmatrix}$$

Our observation model becomes

$$\mathbf{z}_k = h_k(\mathbf{x}_k, \mathbf{n}_k) = H\mathbf{x}_k + \mathbf{n}_k$$

where $\mathbf{z}_k = (z_{k_x}, z_{k_y})^\top$, $\mathbf{n}_k$ is a multivariate Gaussian variable with mean $\{\mathbf{0}\}_{\mathbb{R}^2}$ and covariance R_k (a 2×2 matrix), and H is a matrix given by

$$H = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

It is clear that both our state and observation model are linear.

1.4 Prior and likelihood distributions

Now that we have these two models in place, we can use them to describe what in the probability literature are called the “prior” distribution and the “likelihood” distribution. A good deal of literature on particle filters uses this vocabulary.

The prior distribution is the probability density function that is conditional on the previous time step. It is written as $p(\mathbf{x}_k|\mathbf{x}_{k-1})$. This function answers the question: given a particular value of $\mathbf{x}_{k-1}$ in the past, what is the probability of getting the input $\mathbf{x}_k$ on this time step? This can be answered using our function f_k .

In our example, by looking at f_k we can tell that since $\mathbf{v}_k$ is Gaussian noise with mean $\{\mathbf{0}\}_{R^{n_v}}$ and variance Q_k , we know that $\mathbf{x}_k \sim \mathcal{N}(F\mathbf{x}_{k-1}, GQ_kG^T)$. This is because of the property of a multivariate normal distribution that if $Y = \mathbf{c} + BX$, where $X \sim \mathcal{N}(\mu, \Sigma)$, then $Y \sim \mathcal{N}(\mathbf{c} + B\mu, B\Sigma B^T)$. Therefore, we know that

$$p(\mathbf{x}_k|\mathbf{x}_{k-1}) = \frac{1}{(2\pi)^{N_x/2} \det(GQ_kG^T)^{1/2}} e^{-\frac{1}{2}(\mathbf{x}_k - F\mathbf{x}_{k-1})^T (GQ_kG^T)^{-1} (\mathbf{x}_k - F\mathbf{x}_{k-1})}$$

Because this is painful to write out, it is easier to write

$$p(\mathbf{x}_k|\mathbf{x}_{k-1}) = \mathcal{N}(\mathbf{x}_k; F\mathbf{x}_{k-1}, GQ_kG^T)$$

Here, $\mathbf{x}_k$ is the argument to the pdf of the distribution, $F\mathbf{x}_{k-1}$ is the mean, and GQ_kG^T is the covariance matrix.

The final probability density function we need is the likelihood function. This is written as $p(\mathbf{z}_k|\mathbf{x}_k)$, and it answers the question: given that the state is $\mathbf{x}_k$, how likely is the observation $\mathbf{z}_k$? We can use h_k to answer this question.

In our example, again we can see from the same argument for $\mathbf{x}_k$ that $\mathbf{z}_k \sim \mathcal{N}(H\mathbf{x}_k, R_k)$. Thus, we can write

$$p(\mathbf{z}_k|\mathbf{x}_k) = \mathcal{N}(\mathbf{z}_k; H\mathbf{x}_k, R_k)$$

Being able to switch back and forth between the SDE and probability frameworks will prove very beneficial when reading the literature. It is worth noting that in this framework, the problem of determining state from noisy observations is often stated as trying to find the conditional PDF

$$p(\hat{\mathbf{x}}_k|\mathbf{z}_{1,\dots,k})$$

where $\hat{\mathbf{x}}_k$ is the estimate of the true state $\mathbf{x}_k$ and $\mathbf{z}_{1,\dots,k}$ are the observations up to time k .

2 Kalman Filter variants

We now begin by discussing the Kalman filter. If the state and observation models are linear, as in our example, and the noise is Gaussian, the predicted

state $\hat{\mathbf{x}}_k$ given the observations will be Gaussian as well. The Kalman filter is the optimal predictor of state if the state and observation model are linear, and it also optimally estimates the covariance of the prediction. The derivation will not be presented here; it can be found in many other places such as Wikipedia and other papers.

The algorithm is presented in the following Matlab code:

```
function [ Xhat, P ] = kalman( Z, N, M, X_0, nsteps, f_k_func, F_k_func, ...
                             Q_k_func, h_k_func, H_k_func, R_k_func)

    % Kalman filter estimate
    Xhat = zeros(N, nsteps);

    % Assume initial state is known exactly — can change this later
    Xhat(:,1) = X_0;

    % State prediction covariance, leave at 0 for initial value since
    % state is known perfectly initially
    P = zeros(N,N,nsteps);

    % Now do ensemble Kalman filter based on observations Z
    for k = 2:nsteps
        % Get the linearized state operator from F_k_func
        F_k = F_k_func(k-1, Xhat(:,k-1));

        % Get prediction state from the full model
        m_k_kml = f_k_func(k-1, Xhat(:,k-1));

        % Get the state covariance from Q_k_func
        Q_k = Q_k_func(k-1);

        % Get prediction covariance
        P_k_kml = Q_k + F_k*P(:, :, k-1)*F_k';

        % Get the linearized observation operator from H_k_func
        H_k = H_k_func(k-1, m_k_kml);

        % Get the observation covariance from R_k_func
        R_k = R_k_func(k-1);

        % Get innovation covariance
        S = H_k*P_k_kml*H_k' + R_k;

        % Get optimal Kalman gain
        K = (P_k_kml*H_k')/S;

        % Updated state estimate using the full observation model
        Xhat(:,k) = m_k_kml + K*(Z(:,k) - h_k_func(k, m_k_kml));

        % Updated estimate covariance
        P(:, :, k) = P_k_kml - K*H_k*P_k_kml;
    end
end
```

The main point is that the Kalman filter makes a prediction of the state $m_{k|k-1}$ and covariance $P_{k|k-1}$, then uses this prediction to update and find the

optimal prediction $m_{k|k}$ and the covariance $P_{k|k}$.

2.1 Extended Kalman Filter

If the model is not linear, the derivative of the state model is a linear approximation of f_k near the predicted point

$$F_k = \left. \frac{df_k}{d\mathbf{x}}(\mathbf{x}) \right|_{\mathbf{x}=m_{k-1|k-1}}$$

Likewise, the derivative of the observation model is a linear approximation of h_k near the point $\mathbf{x}_k$.

$$H_k = \left. \frac{dh_k}{d\mathbf{x}}(\mathbf{x}) \right|_{\mathbf{x}=m_{k|k-1}}$$

These approximations can be used with the Kalman filter for a nonlinear model, but because the predicted state need not be Gaussian, the extended Kalman filter is no longer considered optimal in anyway.

3 Particle filter variants

Particle filters attempt to estimate the state of a model by running the same system many times with different paths chosen based on the statistics of the system. Each particle has an associated weight, W , that tells how likely the individual particle is. The particles must be resampled as they have a tendency to become degenerate after a few steps of the model.

The resampling algorithm is shown in the following Matlab code:

```
function [Xstar, wstar, pindex] = resample(X, W, Ns)
    % The resampled particles
    Xstar = zeros(size(X));

    % The resampled weights
    wstar = ones(size(W))/Ns;

    % The indexes of the particles
    pindex = zeros(Ns,1);

    % The cumulative weight of the particles
    c = zeros(Ns,1);

    % For each additional particle
    for i=2:Ns
        % Add to the cumulative weight
        c(i) = c(i-1) + W(i);
    end

    % The index of the particle to resample
    i = 1;
```

```

% The cumulative weight
u = zeros(Ns,1);

% Sample a uniform random between (0,1)
u(1) = rand()/Ns;

% For each particle
for j=1:Ns
    % Compute the cumulative weight of the resampling
    u(j) = u(1) + (j-1)/Ns;

    % Keep moving along the CDF to find the right particle
    while i < Ns && u(j) > c(i)
        i = i + 1;
    end

    % Now set the particle and parent index
    Xstar(:,j) = X(:,i);
    pindex(j) = i;
end
end

```

The main idea is that we sample a $\theta_k \sim U(0,1)$ and then we pick $\mathbf{x}_k^* = \mathbf{x}_n$ so that

$$A^{-1} \sum_{j=1}^n W_j < \theta_k \leq A^{-1} \sum_{j=1}^{n+1} W_j$$

where $A = \sum_{j=1}^{N_s} W_j$.

3.1 Sampling Importance Resampling Filter

The code for the sampling importance resampling filter is shown below

```

function [ Xhat, W ] = particleSir( Z, N, M, X_0, nsteps, Ns, ...
    priorSample_func, likelihoodPdf_func)

% Particle filter estimate with Ns number of ensembles
Xhat = zeros(N, Ns, nsteps);

% Weights for the particles
W = zeros(Ns, nsteps);

% Initialize all particles to the known initial state
for i=1:Ns
    Xhat(:,i,1) = X_0;
    W(i,1) = 1/Ns;
end

% For each time step
for k=2:nsteps
    % Sum of the particle weights
    t = 0;

```



```

% For each ensemble
for i=1:Ns
    % Sample from the prior distribution
    Xhat(:,i,k) = priorSample_func(k-1, Xhat(:,i,k-1));
    % Get the likelihood of the sample and use it as the weight
    W(i,k) = likelihoodPdf_func(k-1, Z(:,k), Xhat(:,i,k));
    % Add to the sum
    t = t + W(i,k);
end

% renormalize weights
W(:,k) = W(:,k)/t;

% Resample the particles
[Xstar, wstar] = resample(Xhat(:, :, k), W(:,k), Ns);

% Use the resampled particles and weights
Xhat(:, :, k) = Xstar;
W(:,k) = wstar;
end
end

```

3.2 Auxiliary Sampling Importance Resampling Filter

The code for the auxiliary sampling importance resampling filter is shown below

```

function [ Xhat, W ] = particleAsir( Z, N, M, X_0, nsteps, Ns, ...
    priorSample_func, likelihoodPdf_func)

Xhat = zeros(N, Ns, nsteps);
W = zeros(Ns, nsteps);

% Initialize all particles to the known initial state
for i=1:Ns
    Xhat(:,i,1) = X_0;
    W(i,1) = 1/Ns;
end

for k=2:nsteps
    t = 0;

    mu_k = zeros(N,Ns);

    for i=1:Ns
        mu_k(:,i) = priorSample_func(k-1, Xhat(:,i,k-1));

        W(i,k) = likelihoodPdf_func(k-1, Z(:,k), mu_k(:,i))*W(i,k-1);

        t = t + W(i,k);
    end

    % renormalize weights
    W(:,k) = W(:,k)/t;
end

```

```

[Xstar, wstar, pindex] = resample(mu_k, W(:,k), Ns);

t = 0;

for j=1:Ns
    Xhat(:,j,k) = priorSample_func(k-1, Xhat(:,pindex(j),k-1));

    W(j,k) = likelihoodPdf_func(k-1, Z(:,k), Xhat(:,j,k))/...
        likelihoodPdf_func(k-1, Z(:,k), mu_k(:,pindex(j)));

    t = t + W(j,k);
end

W(:,k) = W(:,k)/t;
end
end

```